# EXPERIMENT: 8-PUZZLE PROBLEM

## Aim

To write a Python program to solve the 8-Puzzle problem using Artificial Intelligence techniques.

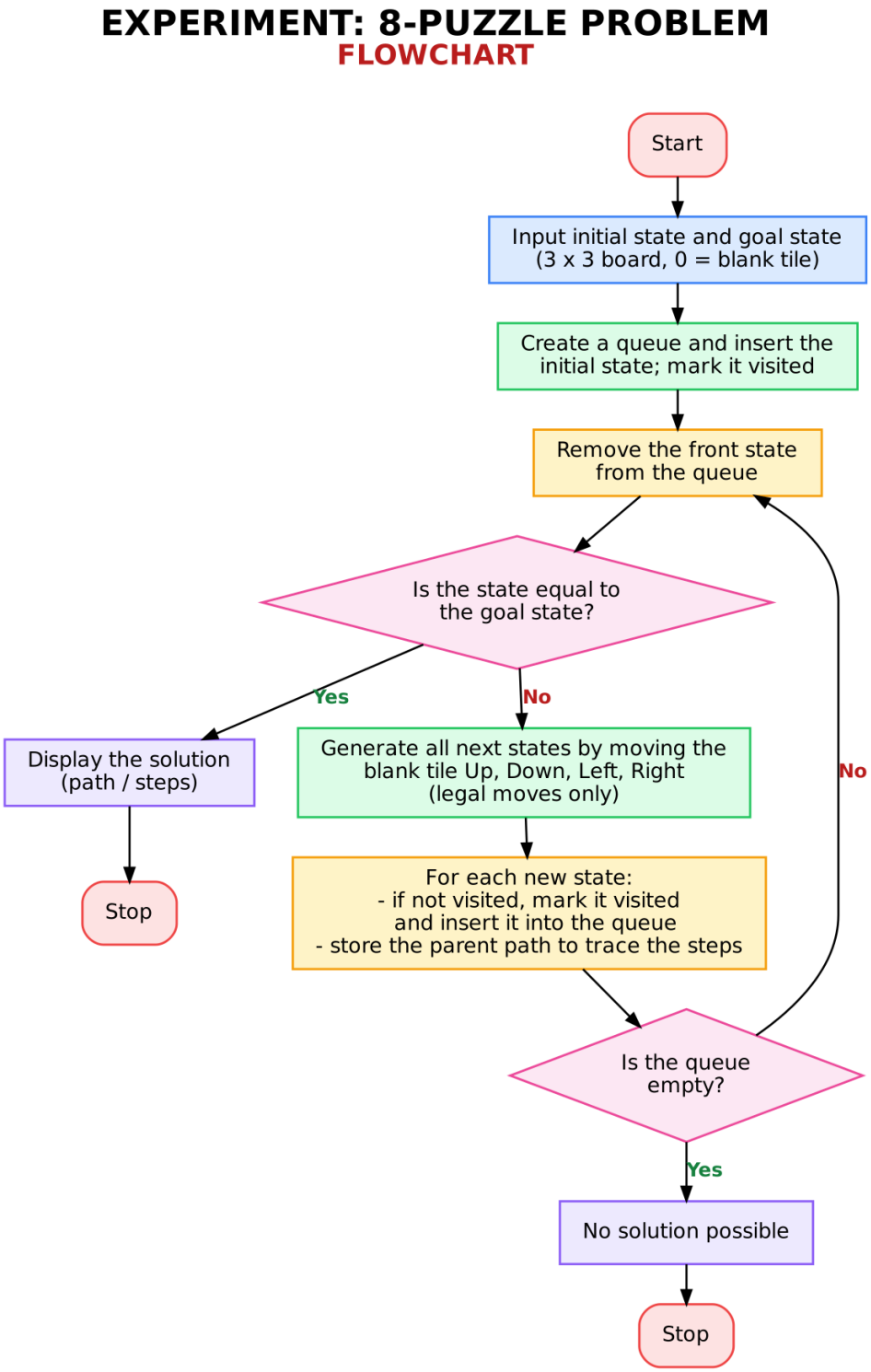
## Objective

- To understand the state-space representation of the 8-Puzzle problem.
- To implement Breadth First Search to find the shortest sequence of moves to the goal state.
- To learn how the legal moves of the blank tile generate new states.

## Algorithm

1. Start.
2. Represent the 3 x 3 board as a tuple of 9 values, where 0 denotes the blank tile.
3. Define the initial state and the goal state.
4. Insert the initial state into a queue and mark it as visited.
5. Remove the front state from the queue.
6. If it is the goal state, display the moves and stop.
7. Otherwise, generate the new states by moving the blank tile Up, Down, Left and Right (legal moves only).
8. Add every unvisited new state to the queue together with its path.
9. Repeat steps 5 to 8 until the goal state is reached or the queue becomes empty.
10. Stop.

## Flowchart:




## Code (Python)

```
from collections import deque

def get_neighbors(state):
    neighbors = []
    i = state.index(0)              # position of the blank tile
    row, col = divmod(i, 3)
```

```
    moves = {"Up": (-1, 0), "Down": (1, 0), "Left": (0, -1), "Right": (0, 1)}

    for move, (dr, dc) in moves.items():
        r, c = row + dr, col + dc
        if 0 <= r < 3 and 0 <= c < 3:
            j = r * 3 + c
            new_state = list(state)
            new_state[i], new_state[j] = new_state[j], new_state[i]
            neighbors.append((tuple(new_state), move))
    return neighbors

def solve_8_puzzle(start, goal):
    queue = deque([(start, [])])
    visited = {start}

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path

        for next_state, move in get_neighbors(state):
            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, path + [(move, next_state)]))
    return None

def print_state(state):
    for i in range(0, 9, 3):
        print(" ".join(str(x) if x != 0 else "_" for x in state[i:i + 3]))
    print()

start = (1, 2, 3,
         5, 0, 6,
         4, 7, 8)
goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

solution = solve_8_puzzle(start, goal)

if solution is None:
    print("No solution exists")
else:
    print("Initial State:")
    print_state(start)
    for step, (move, state) in enumerate(solution, 1):
        print("Step", step, ": Move blank", move)
        print_state(state)
    print("Goal reached in", len(solution), "moves")
```

## Output

**Python Shell - Output**

```
Initial State:
1 2 3
5 _ 6
4 7 8

Step 1 : Move blank Left
1 2 3
_ 5 6
4 7 8

Step 2 : Move blank Down
1 2 3
4 5 6
_ 7 8

Step 3 : Move blank Right
1 2 3
4 5 6
7 _ 8

Step 4 : Move blank Right
1 2 3
4 5 6
7 8 _

Goal reached in 4 moves
```

## Output Result

Thus, the Python program to solve the 8-Puzzle problem was executed successfully, and the goal state was reached in 4 moves.

## Conclusion

The experiment helped in understanding state-space search techniques. The 8-Puzzle problem was solved successfully using Breadth First Search, which explores the states level by level and therefore gives the shortest sequence of moves.